COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - DATA STRUCTURES AND ALGORITHM**	**SECTION:**	CPE21S4
DATE PERFORMED:	10/02/26	**DATE SUBMITTED:**	10/02/26
NAME:	.Del Mundo, Kenneth James Mangcucang, Rhianne L	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO.5.2 **Input Forms and Validation**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

CODE:

adjacent_list.cpp > main()

```cpp
#include <iostream>
using namespace std;

struct adjNode {
    int val, cost;
    adjNode* next;
};

struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;
        return newNode;
    }

    int N;

public:
    adjNode** head;

    DiaGraph(graphEdge edges[], int n, int N) {
        head = new adjNode*[N];
        this->N = N;

        for (int i = 0; i < N; i++) {
            head[i] = nullptr;
        }

        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);

            head[start_ver] = newNode;
        }
    }

    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* current = head[i];

```

```
50
51              while (current != nullptr) {
52                  adjNode* temp = current;
53                  current = current->next;
54                  delete temp;
55              }
56          }
57
58          delete[] head;
59      }
60  };
61
62  void display_AdjList(adjNode* ptr, int i) {
63      while (ptr != nullptr) {
64          cout << "(" << i << ", "
65               << ptr->val << ", "
66               << ptr->cost << ") ";
67
68          ptr = ptr->next;
69      }
70
71      cout << endl;
72  }
73
74  int main() {
75      graphEdge edges[] = {
76          {0, 1, 2},
77          {0, 2, 4},
78          {1, 4, 3},
79          {2, 3, 2},
80          {3, 1, 4},
81          {4, 3, 3}
82      };
83
84      int N = 6;
85      int n = sizeof(edges) / sizeof(edges[0]);
86
```

```
85      int n = sizeof(edges) / sizeof(edges[0]);
86
87      DiaGraph diagraph(edges, n, N);
88
89      cout << "Graph adjacency list" << endl;
90      cout << "(start_vertex, end_vertex, weight):" << endl;
91
92      for (int i = 0; i < N; i++) {
93          display_AdjList(diagraph.head[i], i);
94      }
95
96      return 0;
97  }
```

OUTPUT:

```
PS C:\Users\TIPQC\Documents\LAB 10> cd "c:\Users\TIPQC\Documents
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)

PS C:\Users\TIPQC\Documents\LAB 10>
```

ANALYSIS:

The code shows how a graph can be represented and used in C++. I learned that graphs can be stored using different methods such as an adjacency matrix, adjacency list, and edge list. Each method has its own way of storing the vertices and edges. The output helped me understand how the graph is created and how the connections between vertices are represented.

ILO B
B.1
CODE:

dfs.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <stack>
#include <string>

using namespace std;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    bool operator<(const Edge<T>& e) const {
        return weight < e.weight;
    }

    bool operator>(const Edge<T>& e) const {
        return weight > e.weight;
    }
};

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const {
        return V;
    }

    void add_edge(const Edge<T>& e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V) {
            edge_list.push_back(e);
        }
    }

    vector<Edge<T>> outgoing_edges(size_t v) const {
        vector<Edge<T>> edges_from_v;

        for (const auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.push_back(e);
            }
        }

        return edges_from_v;
```

```
51      }
52
53  private:
54      size_t V;
55      vector<Edge<T>> edge_list;
56  };
57
58  template <typename T>
59  Graph<T> create_reference_graph() {
60      Graph<T> G(9);
61
62      map<unsigned, vector<pair<size_t, T>>> edges;
63
64      edges[1] = {{2, 0}, {5, 0}};
65      edges[2] = {{1, 0}, {5, 0}, {4, 0}};
66      edges[3] = {{4, 0}, {7, 0}};
67      edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
68      edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
69      edges[6] = {{4, 0}, {7, 0}, {8, 0}};
70      edges[7] = {{3, 0}, {6, 0}};
71      edges[8] = {{4, 0}, {5, 0}, {6, 0}};
72
73      for (auto& i : edges) {
74          for (auto& j : i.second) {
75              G.add_edge(Edge<T>{i.first, j.first, j.second});
76          }
77      }
78
79      return G;
80  }
81
82  template <typename T>
83  vector<size_t> depth_first_search(const Graph<T>& G) {
84      stack<size_t> s;
85      vector<size_t> visit_order;
86      set<size_t> visited;
87
88      s.push(1);
89
90      while (!s.empty()) {
91          size_t current_vertex = s.top();
92          s.pop();
93
94          if (visited.find(current_vertex) == visited.end()) {
95              visited.insert(current_vertex);
```

```
96              visit_order.push_back(current_vertex);
97
98              for (auto e : G.outgoing_edges(current_vertex)) {
99                  if (visited.find(e.dest) == visited.end()) {
100                     s.push(e.dest);
101                 }
102             }
103         }
104     }
105
106     return visit_order;
107 }
108
109 int main() {
110     Graph<unsigned> G = create_reference_graph<unsigned>();
111
112     vector<size_t> dfs_visit_order = depth_first_search(G);
113
114     cout << "DFS Order of vertices:" << endl;
115
116     for (auto v : dfs_visit_order) {
117         cout << v << endl;
118     }
119
120     return 0;
121 }
```

OUTPUT:

```
PS C:\Users\TIPQC\Documents\LAB 10> cd "c:\Users\TIPQC\Documents\LAB 10\" ; if ($?) { g++ dfs.cpp -o dfs } ; if ($?) { .\dfs }
DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\Documents\LAB 10>
```

ANALYSIS:

The code demonstrates how Depth-First Search (DFS) works in a graph. It starts from a vertex and continues to explore connected vertices as deeply as possible before going back to another path. The output shows the order in which the vertices are visited. This helped me understand how DFS uses a stack to keep track of the vertices that need to be explored.

B.2

CODE:

bfs.cpp > breadth_first_search<T>(const Graph<T>&)

```cpp
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <queue>
#include <string>

using namespace std;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    bool operator<(const Edge<T>& e) const {
        return weight < e.weight;
    }

    bool operator>(const Edge<T>& e) const {
        return weight > e.weight;
    }
};

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const {
        return V;
    }

    void add_edge(const Edge<T>& e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V) {
            edge_list.push_back(e);
        }
    }

    vector<Edge<T>> outgoing_edges(size_t v) const {
        vector<Edge<T>> edges_from_v;

        for (const auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.push_back(e);
            }
        }

        return edges_from_v;
```

```
51      }
52
53  private:
54      size_t V;
55      vector<Edge<T>> edge_list;
56  };
57
58  template <typename T>
59  Graph<T> create_reference_graph() {
60      Graph<T> G(9);
61
62      map<unsigned, vector<pair<size_t, T>>> edges;
63
64      edges[1] = {{2, 2}, {5, 3}};
65      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
66      edges[3] = {{4, 2}, {7, 3}};
67      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
68      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
69      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
70      edges[7] = {{3, 3}, {6, 4}};
71      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
72
73      for (auto& i : edges) {
74          for (auto& j : i.second) {
75              G.add_edge(Edge<T>{i.first, j.first, j.second});
76          }
77      }
78
79      return G;
80  }
81
82  template <typename T>
83  vector<size_t> breadth_first_search(const Graph<T>& G) {
84      queue<size_t> q;
85      vector<size_t> visit_order;
86      set<size_t> visited;
87
88      q.push(1);
89
90      while (!q.empty()) {
91          size_t current_vertex = q.front();
92          q.pop();
93
94          if (visited.find(current_vertex) == visited.end()) {
95              visited.insert(current_vertex);
```

```
                visited.insert(current_vertex);
                visit_order.push_back(current_vertex);

                for (auto e : G.outgoing_edges(current_vertex)) {
                    if (visited.find(e.dest) == visited.end()) {
                        q.push(e.dest);
                    }
                }
            }
        }

        return visit_order;
    }

    int main() {
        Graph<unsigned> G = create_reference_graph<unsigned>();

        vector<size_t> bfs_visit_order = breadth_first_search(G);

        cout << "BFS Order of vertices:" << endl;

        for (auto v : bfs_visit_order) {
            cout << v << endl;
        }

        return 0;
    }
```

OUTPUT:

```
PS C:\Users\TIPQC\Documents\L
BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\Documents\L
```

ANALYSIS:
The code demonstrates how Breadth-First Search (BFS) traverses a graph. Instead of going deeply into one path, it visits the connected vertices first before moving to the next level. The queue helps maintain the order of the vertices being processed. From the output, I learned how BFS visits the graph layer by layer and avoids visiting the same vertex repeatedly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The most helpful algorithm for this task is Depth-First Search (DFS) because it traverses as deep as possible along a single path before backtracking to explore remaining unvisited vertices. By utilizing a stack mechanism, DFS naturally allows a person to explore one branch completely, hit a dead-end, step back to the previous location, and continue exploring from that point.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The equivalent of DFS in tree traversal strategies is Pre-Order Traversal (Root \rightarrow Left \rightarrow Right), as both methods visit the current node first before diving deep into its children or subtrees.

```
// Tree DFS (Pre-Order Traversal)
void preOrderDFS(TreeNode* root) {
   if (root == nullptr) return;
   std::cout << root->val << " "; // Visit node
   preOrderDFS(root->left);     // Explore left subtree
   preOrderDFS(root->right);    // Explore right subtree
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

A Queue (std::queue) data structure is used to implement Breadth-First Search in the program. It follows a First-In, First-Out (FIFO) approach, which ensures that nodes are processed layer by layer moving outward from the starting vertex.

4. How many times can a node be visited in the BFS?

A node is visited and processed exactly once during the execution of BFS. The algorithm uses a visited set (std::set) to track every vertex added to the queue, preventing any node from being re-visited or processed multiple times.

CONCLUSION

MANGCUCANG

In this activity, I learned how to implement graph data structures in C++ using Adjacency Matrices, Adjacency Lists, and Edge Lists, while mastering fundamental traversal algorithms including Depth-First Search (DFS) using a stack and Breadth-First Search (BFS) using a queue. The procedure demonstrated that DFS explores a path as deep as possible before backtracking, making it ideal for map exploration with dead-ends and equivalent to tree Pre-Order traversal,BFS processes vertices layer by layer to ensure every node is visited exactly once.

DEL MUNDO

In this activity I learned how graphs are built in C++ with ways to represent them. I also learned how Depth-First Search (DFS) and Breadth-First Search (BFS) work when moving through a graph. DFS digs into a path before backtracking while BFS checks each level of vertices one by one. DFS and BFS are the two ways to explore graphs. Overall this activity helped me understand how graphs and how their traversal methods work and how graphs can help solve problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.